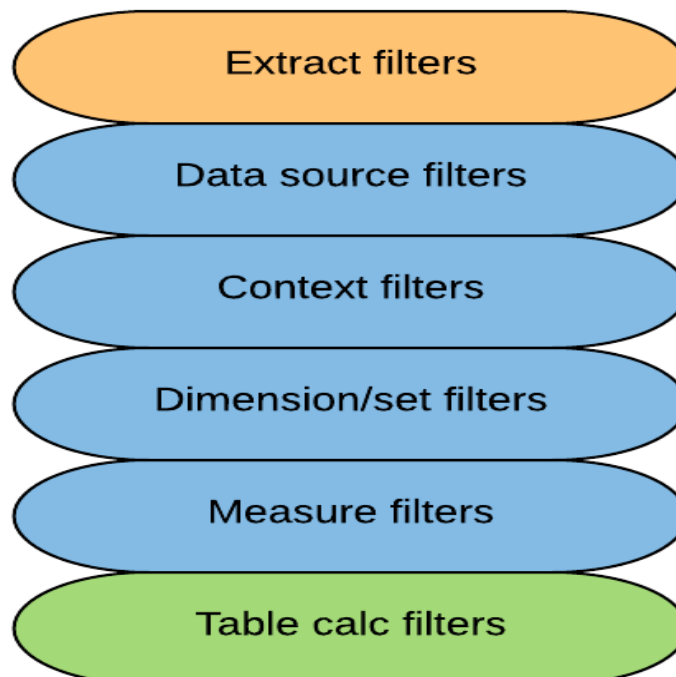


Best Practices for Designing Efficient Tableau Workbooks

- Extracts are a quick and easy way to make most workbooks run faster.
- The majority of slow dashboards are caused by poor design – in particular, too many charts on a single dashboard, or trying to show too much data at once.
- Strings and dates are slow, numbers and Booleans are fast.
- Start performance Recording is the option under Setting and Performance of Help Menu. Used to record the performance of Tableau workbook and generates the result in the form of workbook.
- The Log file is quite verbose and is written as JSON encoded text, so a good text editor such as Notepad++ or Sublime is recommended to wrangle it.
- TabMon - It is an open source cluster monitor for Tableau Server that allows you to collect performance statistics over time.
- TabJolt - It is a “point-and-run” load and performance testing tool specifically designed to work easily with Tableau Server.

- **Filter types & Filter priority :**

In Tableau, filters are applied in the following order:



- Extract filters - These filters are only applicable when using data extracts, but in that case are logically applied before all other filters.
- Data source filters - Data source filters are the highest level of filter available on live connections.
- Cross data source filters - Cross data source filtering is a new feature in Tableau 10. It allows a filter to be applied to multiple data sources that have one or more fields in common.
- Date filters – Discrete date filters, Range of Date, Relative Date filters.
- Parameters are single-value only – you cannot use them if you want the user to select multiple values.
- Once defined, a calculated field is available across the entire workbook as long as the worksheets are using the same data source.
- There are four different calculation types in Tableau:
 - Row-level calculations,
 - Aggregate calculations,
 - Table calculations, and
 - Level of Detail expressions.

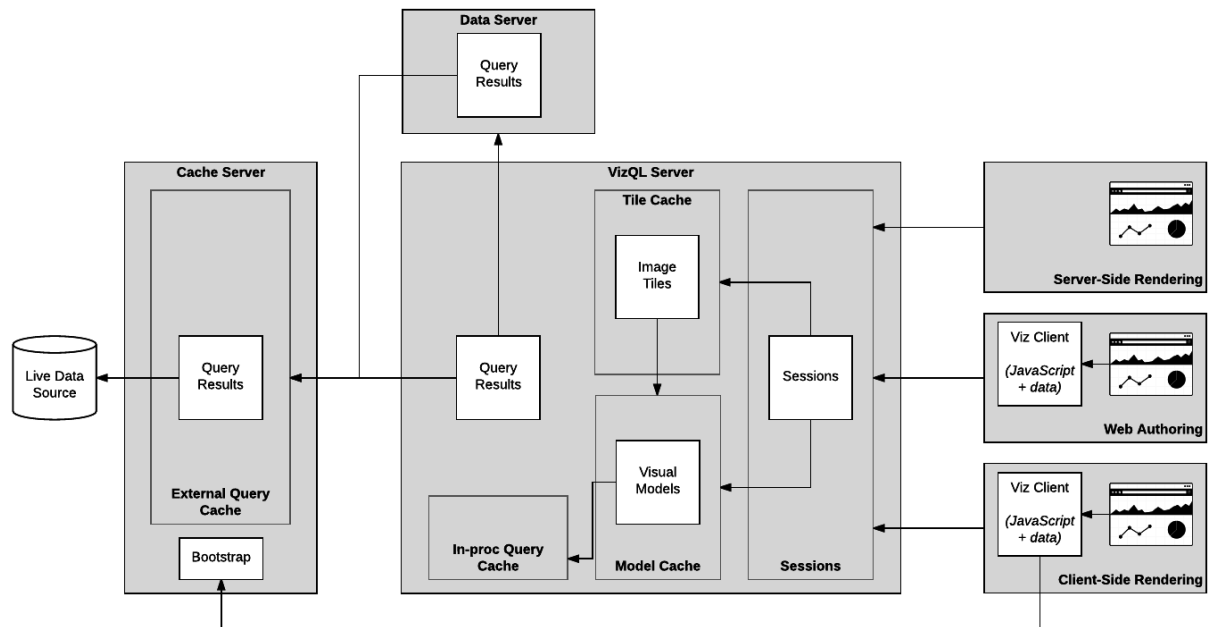
Calculation Types :

- Row-level and aggregate calculations - The YEAR calculation is a row-level calculation and the SUM(SALES) is an aggregate calculation.
- How a calculated field works as a query at background ?
- calculated field Profit Ratio defined as $\text{SUM}([\text{Profit}])/\text{SUM}([\text{Sales}])$:

```
SELECT DATEPART(year,[OrdersFact].[Order Date]) AS [yr:Order Date:ok],
SUM([OrdersFact].[Profit]) AS [TEMP(Calculation_0260604221950559)
(1796823176)(0)],
SUM([OrdersFact].[Sales]) AS [TEMP(Calculation_0260604221950559)
(3018240649)(0)]
FROM [dbo].[OrdersFact] [OrdersFact]
GROUP BY DATEPART(year,[OrdersFact].[Order Date])
```

- ATTR() is a powerful function, often used as an aggregation for categorical dimensions.
- Level of detail calculations - Level of detail (LOD) expressions allow you to define the level of detail at which a calculation should be performed, independently of the viz LOD.

- Table calculations – unlike row-level and aggregate calculations – are not executed by the database but rather calculated by Tableau on the query result set.
- Running the Queries in parallel mode will increase the performance.
- Tableau Server :
Caching in Tableau Server has multiple layers:



- client-side rendering produces smoother, more responsive user interactions and places less workload on the Tableau Server.
- Three types of cache – Model, In-proc, External cache.
- In Tableau Desktop the external query cache data is included when the file is saved as a packaged workbook.
- Initial SQL is executed only once when the workbook is opened.
- One of the powerful features of Tableau is its ability to connect to data across many different platforms. Broadly speaking these platforms can be characterised as one of the following:
 1. File-based data sources – such as Excel and CSV;
 2. Relational database data sources – such as Oracle, Teradata and SQL Server, as well as specialised analytic appliances such as HP Vertica, IBM Netezza, etc;
 3. OLAP data sources – such as Microsoft Analysis Services and Oracle Essbase;

4. “Big data” data sources – such as Hadoop;

5. Cloud-based data sources – such as Salesforce, Google, etc.

- Tableau keeps shadow extracts for the five most-recently used file data sources in this directory, deleting the least recently used file when a new one is created.
- RDBMS systems come in two main flavours – row-based or columns-based. Row-based storage layouts are well-suited for OLTP-like workloads which are more heavily loaded with interactive transactions. Column-based storage layouts are well-suited for analytic workloads (e.g., data warehouses) which typically involve highly complex queries over large sets of data.
- Having NULL values in dimension columns can reduce the effectiveness of queries.
- Partitioning is a recommended strategy for large data volumes and is transparent to Tableau.
- [OLAP](#) :

Tableau supports several OLAP data sources:

1. Microsoft Analysis Services
2. Microsoft PowerPivot (both PowerPivot for Excel and PowerPivot for SharePoint)
3. Oracle Essbase
4. SAP Business Warehouse
5. Teradata OLAP

- [Big Data](#) :

Big data is a heavily overloaded term in the world of data analysis. In Tableau 10 there are four supported Hadoop distributions that support Hive and/or Impala connections:

1. Amazon EMR
 - a. HiveServer
 - b. HiveServer2
 - c. Impala
2. Cloudera Hadoop
 - a. HiveServer
 - b. HiveServer2
 - c. Impala
3. Hortonworks Hadoop Hive
 - a. HiveServer

- b. HiveServer2
- c. Hortonworks Hadoop Hive
- 4. MapR Hadoop Hive
 - a. HiveServer
 - b. HiveServer2
- 5. Spark SQL
 - a. SharkServer *
 - b. SharkServer2 *
 - c. SparkThriftServer

** Note that SharkServer and SharkServer2 connections are provided for your use, but are not supported by Tableau.*

- Impala is generally much faster than Hive and Spark is proving to be faster still.

- Cloud :

Tableau currently supports the following cloud data sources:

1. Salesforce.com
2. Google Analytics
3. oData (including Windows Azure Marketplace DataMarket)

- Tableau also supports the following cloud-based data platforms:

1. Amazon Aurora
2. Amazon Redshift
3. Amazon RDS
4. Google BigQuery
5. Google Cloud SQL
6. Google Sheets
7. Microsoft Azure SQL Data Warehouse
8. Snowflake

- Finally, Tableau also provides a generic data connector for importing data from web-based services that publish data in JSON, XML, or HTML formats – the Web Data Connector.
- Salesforce.com balances transaction loads by imposing two types of limits: Concurrent API Request Limits and Total API Request Limits.
- Using a web data connector, you can create and use a connection to almost any data that is accessible over HTTP.

- An extract is:
 - A persistent cache of data that is written to disk and reproducible;
 - A columnar data store – a format where the data has been optimised for analytic querying;
 - Completely disconnected from the database during querying. In effect, the extract is a replacement for the live data connection;
 - Refreshable, either by completely regenerating the extract or by incrementally adding rows of data to an existing extract;
 - Architecture-aware – unlike most in-memory technologies it is not constrained by the amount of physical RAM available;
 - Portable – extracts are stored as files so can be copied to a local hard drive and used when the user is not connected to the corporate network. They can also be used to embed data into packaged workbooks that are distributed for use with Tableau Reader;
 - Often much faster than the underlying live data connection.
- Finally, extracts cannot be created over OLAP data sources such as SQL Server Analysis Services, or Oracle Essbase. The exception to this rule is that you can create extracts from SAP BW.

- **When to use extracts? When to use live connections?**

Like everything, there's a time and a place for data extracts. The following are some scenarios where extracts may be beneficial:

1. Slow query execution
 2. Offline analysis
 3. Packaged workbooks for Tableau Reader/Online/Public
 4. Additional functionality
 5. Data security
- Extracts are very powerful, but they are not a silver bullet for all problems. There are some scenarios where using extracts might not be appropriate:
 1. Real-time data
 2. Massive data
 3. Pass-through RAWSQL functions
 4. Robust user-level security
 - Tableau also provides an application programming interface (API) to enable developers to directly create a Tableau Data Extract (TDE) file.

- Creating aggregate extracts is a very powerful technique when you have a large amount of base data but you need to create summary views that query across the whole data set.
- You can choose two refresh schedules for a single extract:
 1. An incremental refresh just adds rows, and does not include changes to existing rows.
 2. A full refresh discards the current extract and regenerates a new one from scratch from the data source.

Extract maintenance :

The Maintenance screens show what Background tasks are currently running, as well as those that have run for the past 12 hours.

- The default VizQL session timeout limit is 30 minutes. Even if a VizQL session is idle, it is still consuming memory and CPU cycles. If you can make do with a lower limit, use tabadmin to change the vizqlserver.session.expiry.timeout setting
- From Tableau 10 and onwards, Tableau Server is only available as a 64-bit application.
- TabMon is an excellent tool for collecting utilisation data that can help your capacity planning process.
- Tableau heavily utilises JavaScript so the speed of the JavaScript interpreter in the browser has an impact of the speed of rendering.

Conclusion :

So again, here are the key points I hope you take away from reading this document:

- ✓ There is no silver bullet. Performance can be slow for a number of reasons. Collect data to help identify where the time is going. Then focus your improvements in the most expensive area, then move to the next area, and so on. Stop when it's fast enough or the effort/reward ratio is no longer viable.
- ✓ Many slow dashboards are caused by poor design. Keep it simple. Don't try to show too much at once and use guided analysis designs where possible.
- ✓ Don't work with data you don't need. Use filters, hide unused fields and aggregate.
- ✓ Try not to fight against the data engine. It's smart and it's trying to help you, so trust that it will generate efficient queries.
- ✓ The cleaner your data is and the better it matches the structure of your questions, the faster your workbooks will run and the happier your life will be.
- ✓ Extracts are a quick and easy way to make most workbooks run faster. If you don't need real-time data and aren't working over billions of rows of data, you should try them.
- ✓ Strings and dates are slow, numbers and Booleans are fast.
- ✓ Although this document is based on the best advice from a lot of smart people, its recommendations are just that – recommendations. You will need to test which ones will improve performance in your specific case.
- ✓ When you are working with small data volumes much of this won't matter – you can just brute force through bad technique. But it doesn't hurt to follow these recommendations in all your workbooks as you never know when your data will grow.
- ✓ Practice makes perfect.

Now go forth, and make lots of efficient workbooks!